

Contents

1	Programming using Matlab	1
1.1	Introduction	1
1.2	Vectors and matrices	5
1.3	Character strings, cell arrays, structures	15
1.4	Statement level control structures	19
1.4.1	Logical expressions	19
1.4.2	Conditional statements	23
1.4.3	Loop statements	24
1.5	Functions	25
1.5.1	Built-in functions	25
1.5.2	Function files	27
1.5.3	Anonymous functions	29
1.5.4	Passing functions as input	29
1.6	Graphics	30
1.6.1	2-D graphics	30
1.6.2	3-D graphics	33
1.7	Efficiency	37
1.7.1	Vectorising loops	37
1.7.2	Preallocating arrays	37

Chapter 1

Programming using Matlab

1.1 Introduction

In this course, we will use Matlab to carry out all of the programming exercises. Matlab is a software package designed for numerical computation and visualisation. The symbolic toolbox can do symbolic algebra as well. MATLAB = MATrix LABoratory. As the name suggests, the building blocks of Matlab are matrices.

Matlab provides an interactive environment. In a default setup, there are four main windows: Command Window, Current Directory, Workspace, and Command History.

- Command Window: This is the main window where all commands including those for running user-written programs are typed in. The MATLAB command prompt is `>>`. E.g.

```
>> help help
>> help
>> help quad
```

- Current Directory: This is where all your files from the current directory are listed. To change directories, click on the menu bar or in the Command Window:

```
>> cd directory_name
```

To use a file from another directory, use `addpath`.

- Workspace: This window lists all variables you have generated in this session, together with their type and size. Double click on a variable in this window to view its value. Right click for other options.

- **Command History:** This window records all commands typed in the Command Window. Double click on a command to execute it again. You can also use the up-arrow on the keyboard.

A comprehensive set of demonstrations is available by typing the command

```
>> demo
```

and following the instructions.

The fundamental data type in MATLAB is the array which includes integers, real numbers (which are set to be of double precision), matrices, character strings, structures, and cells.

In contrast to other programming languages, no declarations of data type or data object are needed. For vectors and arrays, the dimension statements are not required either.

Matlab stored computed values in variables. You don't have to declare variable name as the beginning of the script, but an assignment statement is needed. Variable names are case-sensitive, can contain up to 31 characters, and must start with a letter. These are allowable:

```
My_Variable, my_variable, Math2301.
```

These are not allowable:

```
my-variable, 2301math, @UNSW
```

There is a reserved word list. To check if a variable name *var* has been used or is in the reserved list, type in the following command

```
>> exist('var')
```

The following table lists some special variables: If you assign a new value to one

Variable name	Meaning
ans	default variable name used for results
pi	π
eps	smallest number such that when added to 1 gives a number greater than 1 on the computer
inf, Inf	∞
NaN, nan	not a number, e.g. $0/0$
i, j	imaginary number $\sqrt{-1}$

of the special variables above, its prior value is overwritten and lost.

To clear a variable from the memory, you type in

```
>> clear variable_name
```

To clear all variables from the memory, type in

```
>> clear all
```

Sometimes, you don't want too much information to be printed out on the screen. A semicolon at the end of a command suppresses the screen output, except for graphics and on-line help commands. The command `more on` directs MATLAB to show one screen of output at a time. To turn it off, type `more off`. Though MATLAB performs computations in double precision, the default appearance is a scaled fixed point format with 5 digits (format short). The command

```
>> format compact
```

suppresses blank lines in the output, thus allowing more information to be displayed on the screen.

MATLAB provides its own built-in editor with an integrated debugger. You can use this editor to create and save your own programs in files. Alternatively, you can use any text editor, like `vi` or `emacs`. MATLAB has 3 types of files:

- M-files: standard ASCII text files, with a `.m` extension to the file name. There are 2 types of M-files: script files and function files.
- Mat-files: binary data files with a `.mat` extension to the file name.
- Mex-files: MATLAB callable Fortran and C programs, with a `.mex` extension to the file name.

A MATLAB script is a text file containing MATLAB commands, and having a `.m` extensions. To execute commands in a script, simply type the name of the file, without the `.m` extension, at the MATLAB prompt. Running a script file is equivalent to typing all the commands (in that file) one by one at the MATLAB prompt. The advantage is obvious: script files are easy to modify. The script file has to be in the current directory, or in a directory of the MATLAB search path. The command `what` can be used to check if the file is in the current directory. A path can be added by the command `addpath`. As a warning, the name of a script file cannot be the same as the name of a variable it computes.

Example 1.1.1 Write a MATLAB script, named `area.m`, to compute the areas of the circles of radii $\pi^{1/3}$ and $\pi^{2/3}$. On running the script should print out the following line (with no other outputs on the screen)

```
Radius      Area
```

```
. . .      . . .
```

.

where ... contains the appropriate value.

```
% MATLAB script to compute the areas of circles of
% radii  $\pi^{1/3}$  and  $\pi^{2/3}$ .
disp([' Radius Area'])
disp(' ')
r =  $\pi^{1/3}$ ; A =  $\pi*r^2$ ;
disp([r, A])
r =  $\pi^{2/3}$ ; A =  $\pi*r^2$ ;
disp([r, A])
```

To execute, type in the MATLAB command window: `area`

If some variables are defined, they can be saved for next sessions, using the command `save filename`. A file named `filename.mat` is created in the current directory. The command `load filename` reloads the values in the next session.

Example 1.1.2 Try the following commands on the command window.

```
>> a = pi; b = a^2; c = a+b;
>> save a_value a      % save value of a only
>> save ab_value a b   % save value of a and b
>> save all_values     % save all values in session
>> clear all
>> a, b, c             % What do you expect to see?
>> load ab_value       % Do not include .mat
>> a, b, c             % What do you expect to see?
>> load all_values
>> c
```

For reading and writing formatted binary and text files, the following commands should suffice `fopen`, `fprintf`, `fscanf`, and `fclose`.

One of the following commands opens a file named `MyFile`.

```
% Open file named MyFile for read access.
% File must exist before command is executed
>> fid = fopen('MyFile');
% or
>> fid = fopen('MyFile','r');
% Open or create file for writing,
% existing contents overwritten
```

```
>> fid = fopen('MyFile','w');
% Open or create file for writing,
% append data to end of file
>> fid = fopen('MyFile','a');
% Open file for reading and writing
>> fid = fopen('MyFile','r+');
% Open or create file for reading and writing
>> fid = fopen('MyFile','w+');
% Open or create file for reading and appending
>> fid = fopen('MyFile','a+');
```

The file identifier `fid` (an integer) is used as the first argument to `fprintf` and `fclose`. It has the value `-1` when a file cannot be opened.

```
>> x = [1:10];
>> y = sqrt(x);
>> fid = fopen('square_root','w');
>> fprintf(fid,' Square Root\n ');
>> fprintf(fid,'\n');
>> fprintf(fid,' %3i %8.2f\n',[x;y]);
>> fclose(fid);
```

To read and write binary data to a file, use `fread` and `fwrite`.

1.2 Vectors and matrices

A row vector is defined by listing numbers which are separated by either commas or spaces. A column vector has entries separated by semicolons or new lines.

```
>> v1 = [ 1 2, 3 4 ]
v1 =
     1     2     3     4
>> v2 = [ 3 ; 4
          6 ]
v2 =
     3
     4
     6
>> v1(3), v2(2)
ans =
     3
ans =
     4
```

Row and column vectors are one-dimensional arrays. Matrices are two-dimensional arrays. It is not necessary to declare the dimension of a matrix.

```
>> A = [ 1 2 3
         4 5 6 ]
A =
     1 2 3
     4 5 6
>> B = [ 3 7 9 ; 2 6 4 ]
B =
     3 7 9
     2 6 4
```

If it is not possible to type the entire input on the same line, use an ellipsis (three dots) and continue the input on the next line.

```
>> A = [ 4 7 9 7 ...
         8 9 ]
A =
     4 7 9 7 8 9
```

The **size** command gives the number of rows and columns of an array. The **length** command gives the maximum value of these two numbers.

```
>> A = [1 8 0 3 ; 2 4 9 6 ; 0 3 7 9];
>> size(A)
ans =
     3 4
>> length(A)
ans =
     4
>> [m n] = size(A) % assign these values to variables
                        % m and n for later access
m =
     3
n =
     4
>> size(A,1)
ans =
     3
>> size(A,2)
ans =
     4
```



```

>> a = [2 3 4 5 9];
>> size(a)
ans =
     1     5
>> length(a)
ans =
     5
>> B = [] % empty array
B =
     []
>> size(B)
ans =
     0     0
>> a(3)
ans =
     4
>> a(2) = 0
>> a
a =
     2     0     4     5     9
>> A(2,3), A(5)
ans =
     9
ans =
     4
>> A(3,1) = 8
A =
     1     8     0     3
     2     4     9     6
     8     3     7     9

```

Example 1.2.1 Solve the equation $A\mathbf{x} = \mathbf{b}$ where

$$A = \begin{bmatrix} 1 & 8 & 3 \\ 2 & 4 & 6 \\ 8 & 3 & 9 \end{bmatrix} \text{ and } \mathbf{b} = \begin{bmatrix} 2 \\ 3 \\ 5 \end{bmatrix}$$

```

>> A = [1 8 3 ; 2 4 6 ; 8 3 9]; b = [2 ; 3 ; 5];
>> x = inv(A)*b
x =
    0.1500
    0.0833
    0.3944

```

```
>> x = A\b % Almost the same as inv(A)*b but
x = % faster and more numerically stable
    0.1500
    0.0833
    0.3944
```

Notes:

```
>> c = [1 3 4];
>> x = A\c
??? Error using ==> mldivide
Matrix dimensions must agree.
>> x = A\c'
x =
   -0.1500
   -0.0833
    0.6056
```

Given a scalar a (real or complex) and two matrices A and B . The following operations are available under some appropriate conditions.

Command	Meaning	Condition
$C = a + A$	$C_{ij} = a + A_{ij}$	$\text{size}(A) = \text{size}(B)$ $\text{size}(A) = \text{size}(B)$ $n = \text{size}(A, 2) = \text{size}(B, 1)$ $\text{size}(A, 1) = \text{size}(A, 2)$
$C = a - A$	$C_{ij} = a - A_{ij}$	
$C = a * A$	$C_{ij} = a * A_{ij}$	
$C = A/a$	$C_{ij} = A_{ij}/a$	
$C = A + B$	$C_{ij} = A_{ij} + B_{ij}$	
$C = A - B$	$C_{ij} = A_{ij} - B_{ij}$	
$C = A * B$	$C_{ij} = \sum_{k=1}^n A_{ik} B_{kj}$	
$C = A^n$	$C = \underbrace{A * \dots * A}_n$ n times	
$C = A'$	$C_{ij} = A_{ji}$	
$C = \text{inv}(A)$	$C = A^{-1}$	
$C = A/B$	$C = AB^{-1}$	$\text{size}(A, 1) = \text{size}(A, 2)$ $\text{size}(A, 2) = \text{size}(B, 1) = \text{size}(B, 2)$
$C = A \setminus B$	$C = A^{-1}B$	$\text{size}(A, 1) = \text{size}(A, 2) = \text{size}(B, 1)$

Consider the following commands

```
>> v = [3:9] % increment by 1
v =
    3    4    5    6    7    8    9
>> v = 3:2:9 % increment by 2
v =
```

```

3 5 7 9
>> v = [3:2:8]
v =
3 5 7          % final value does not exceed 8
>> v = [8:-2:3] % negative increment
v =
8 6 4          % final value does not go below 3
>> v = [5:2]
v =
Empty matrix: 1-by-0
>> v = 2:-1:3
v =
Empty matrix: 1-by-0

```

The command `linspace(a,b,n)` generates a vector of n linearly equally spaced points between (and including) a and b . If n is not specified, i.e. `linspace(a,b)`, the default value is $n = 100$.

Example 1.2.2 Generate a vector of 5 equally spaced points between and including 1 and 10.

```

>> linspace(1,10,5)
ans =
    1.0000    3.2500    5.5000    7.7500   10.0000

```

This is the same as

```

>> [1:(10-1)/(5-1):10] % [a:(b-a)/(n-1):b]
ans =
    1.0000    3.2500    5.5000    7.7500   10.0000

```

If a is a scalar, and A and B are two arrays of the same size, then the element-by-element operations give the following arrays:

Operation	Meaning
$C = a./A$	$C_{ij} = a/A_{ij}$
$C = A.\backslash a$	$C_{ij} = a/A_{ij}$
$C = A.^a$	$C_{ij} = A_{ij}^a$
$C = a.^A$	$C_{ij} = aA_{ij}$
$C = A.*B$	$C_{ij} = A_{ij}B_{ij}$
$C = A./B$	$C_{ij} = A_{ij}/B_{ij}$
$C = A.\backslash B$	$C_{ij} = B_{ij}/A_{ij}$
$C = A.^B$	$C_{ij} = A_{ij}^{B_{ij}}$

```
>> v = 1:5
v =
    1  2  3  4  5
>> 2.^v
ans =
    2  4  8 16 32
>> v.^2
ans =
    1  4  9 16 25
>> v^2
??? Error using ==> mpower
    Matrix must be square.
```

To access parts of a matrix, consider the following commands:

```
>> A = [2 5 -1 7 ; 8 0 6 3 ; 9 8 -2 1 ; 3 7 9 2]
A =
    2  5 -1  7
    8  0  6  3
    9  8 -2  1
    3  7  9  2
>> B = A(2:4,3:4)
B =
     6  3
    -2  1
     9  2
>> C = A(1:2:4,:)
C =
    2  5 -1  7
    9  8 -2  1
>> A(2:4,3:4) = 0
A =
    2  5 -1  7
    8  0  0  0
    9  8  0  0
    3  7  0  0
>> A(1:2:end,1) = -1
A =
   -1  5 -1  7
    8  0  0  0
   -1  8  0  0
    3  7  0  0
```

Using an array to refer to part of a matrix:

```
>> j = [1 3 4];
>> A(3,j)
ans =
    -1     0     0
```

We can stack small matrices one on top of each others.

```
>> A = [1 -1 ; 2 2]; B = [2 1 ; 0 0]; C = [1:4 ; 5:8];
>> E = [A B ; C]
E =
     1    -1     2     1
     2     2     0     0
     1     2     3     4
     5     6     7     8
>> D = [1:5, 5:8];
>> F = [A B ; D]
??? Error using ==> vertcat
All rows in the bracketed expression must have the
same number of columns.
```

Zeros (or ones) arrays are arrays that contains all zeros (or all ones).

```
>> zeros(3)
ans =
     0     0     0
     0     0     0
     0     0     0
>> zeros(2,3)
ans =
     0     0     0
     0     0     0
>> A = [1 2 5 7 ; 2 4 8 1];
>> zeros(size(A))
ans =
     0     0     0     0
     0     0     0     0
```

Replacing **zeros** by **ones** gives similar matrices with entries 1.

We can define identity matrices as follows

```
>> eye(2)
ans =
     1     0
```

```

    0 1
>> eye(3,2)
ans =
    1 0
    0 1
    0 0
>> eye(2,3)
ans =
    1 0 0
    0 1 0
>> A = [1 2 5 7 ; 2 4 8 1];
>> B = eye(size(A))
B =
    1 0 0 0
    0 1 0 0

```

We can access diagonals of a matrix by the `diag` command. If A is a matrix then `diag(A,k)`, where k is an integer, produces a column vector whose components are entries on the k th diagonal of A .

```

>> A = [1 3 5 ; 2 4 6]; v1 = diag(A), v2 = diag(A,1), ...
    v3 = diag(A,2), v4 = diag(A,3), v5 = diag(A,-1)
v1 =
    1
    4
v2 =
    3
    6
v3 =
    5
v4 =
    Empty matrix: 0-by-1
v5 =
    2

```

The same command can be use to create diagonal matrices. If v is a vector, then `diag(v,k)` creates a square matrix of appropriate size whose k th diagonal is v .

```

>> v = [1:3]; v1 = diag(v), v2 = diag(v,1), v3 = diag(v,-2)
v1 =
    1 0 0
    0 2 0
    0 0 3
v2 =

```

```

    0 1 0 0
    0 0 2 0
    0 0 0 3
    0 0 0 0
v3 =
    0 0 0 0 0
    0 0 0 0 0
    1 0 0 0 0
    0 2 0 0 0
    0 0 3 0 0

```

Sparse matrices are matrices which contain a lot of zeros.

```

>> A = [0 0 1 0 0 ; -2 0 3 0 0 ; 0 0 -5 0 0]
A =
    0 0 1 0 0
   -2 0 3 0 0
    0 0 -5 0 0
>> nnz(A) % number of non-zero entries of A
ans =
    4
>> B = sparse(A)
B =
    (2,1) -2
    (1,3)  1
    (2,3)  3
    (3,3) -5

```

The command `full(B)` reproduces a matrix identical to *A*. Arithmetic and indexing operations can be applied to sparse matrices or to a mixture of sparse and full matrices.

```

>> clear all % clear all existing variables from memory
>> clc       % clear the Command Window
>> A = [2 0 5 0 0 ; 0 0 0 4 0 ; 0 0 0 1 0]; B = sparse(A);
>> nnz(A); v = [9 11 8];
>> who
Your variables are:
A B ans v
>> whos A
Name Size Bytes Class
A 3x5 120 double array
Grand total is 15 elements using 120 bytes
>> whos

```

```

Name      Size  Bytes  Class
   A       3x5    120   double array
   B       3x5     72   double array (sparse)
  ans      1x1     8    double array
   v       1x3    24    double array
Grand total is 23 elements using 224 bytes
>>
>> A = eye(200); B = sparse(A);
>> whos
Name      Size      Bytes    Class
   A     200x200  320000   double array
   B     200x200   3204    double array (sparse)
Grand total is 40200 elements using 323204 bytes

```

For more information on how sparse matrices are stored, read J.R. Gilbert, C. Moler, and R. Schreiber, Sparse matrices in MATLAB: Design and implementation. http://www.mathworks.com/help/pdf_doc/otherdocs/simax.pdf.

Example 1.2.3 A sparse matrix B of size 4×4 with nonzeros entries $B(2,1) = -2$, $B(1,2) = 1$, $B(4,2) = 1$, $B(2,3) = 3$, $B(3,3) = -5$, $B(4,4) = 1$ can be created as follows:

```

>> a = [-2 1 1 3 -5 1]; % non-zero entries of B
>> r = [2 1 4 2 3 4]; % row indices
>> c = [1 2 2 3 3 4]; % column indices
>> B = sparse(r,c,a,4,4)
B =
    (2,1) -2
    (1,2) 1
    (4,2) 1
    (2,3) 3
    (3,3) -5
    (4,4) 1

```

If A is a given matrix, the command `sA = sparse(A)` produces a matrix sA which is a sparse version of A .

Sparse matrices are naturally defined in terms of their diagonals using `spdiags` command.

```

>> e = ones(5,1); % some none-zero entries
>> A = spdiags([-e 2*e -e],... % 0 -> diagonal, -1 -> subdiagonal
               [-1 0 1],5,5); % 1 -> superdiagonal
>> full(A)

```



```
ans =
    2 -1  0  0  0
   -1  2 -1  0  0
    0 -1  2 -1  0
    0  0 -1  2 -1
    0  0  0 -1  2
```

1.3 Character strings, cell arrays, structures

In MATLAB a **string** is an array of ASCII character codes. (ASCII = American Standard Code for Information Interchange).

```
>> s = 'Hello world!'
s =
    Hello world!
>> whos s
    Name  Size  Bytes  Class
    s     1x12    24   char array
Grand total is 12 elements using 24 bytes
>> isstr(s), isstr(5), isstr('5') % check if the variable
ans = % is a string
    1
ans =
    0
ans =
    1
```

Since strings are arrays, they can be manipulated with all array manipulation tools.

```
>> a = 'Hello'; b = 'world'; c = '!';
>> [a b c]
ans =
    Helloworld!
>> [a ' ' b c]
ans =
    Hello world!
>> a(1:4)
ans =
    Hell
>> b(end:-1:1)
ans =
    dlrow
>> u = 'I don''t know how' % single quotes within character are
```

```
u =                                % symbolised y two consecutive quotes
    I don't know how
```

Strings can have multiple rows, but each row must have equal number of columns.

```
>> v = ['Joshua' ; 'Jack ' ; 'Ben ']
v =
    Joshua
    Jack
    Ben
```

Artificial creation of blank spaces can be avoided by using char command.

```
>> student = char('Joshua', 'Jack', 'Ben')
student =
    Joshua
    Jack
    Ben
```

Horizontal concatenation of string arrays having the same number of rows is accomplished by the function strcat.

```
>> strcat('Adam', ' Smith')
ans =
    Adam Smith
>> students = char('Joshua', 'Jack', 'Ben');
>> surnames = char(' Waugh', ' Warne', ' Bevan');
>> v = strcat(students, surnames)
v =
    Joshua Waugh
    Jack Warne
    Ben Bevan
>> whos v
    Name      Size      Bytes      Class
    v         3x12       72       char array
Grand total is 36 elements using 72 bytes
```

A **cell array** is an array that can contain any MATLAB object, e.g., an array of numbers, logicals, a string, etc.

```
>> A = {[1 2 ; 3 4], 'MATH2301', 1 + 2i, 1 > 0}
A =
    [2x2 double] 'MATH2301' [1.0000 + 2.0000i] [1]
>> whos A
    Name      Size      Bytes      Class
    A         1x4       305      cell array
Grand total is 18 elements using 305 bytes
```

The curly braces indicate that the expression represents a cell rather than, e.g., a numerical value. We can preallocate a variable as a cell array, using the command `cell`.

```
>> B = cell(2,2)
B =
    [] []
    [] []
>> B{1,1} = ones(1,3); B{1,2} = rand(1,2);
>> B{2,1} = 'MATLAB'; B{2,2} = A
B =
    [1x3 double] [1x2 double]
    'MATLAB' {1x4 cell }
```

To display the content of a cell array, use `celldisp`.

```
>> celldisp(A)
A{1} =
    1 2
    3 4
A{2} =
    MATH2301
A{3} =
    1.0000 + 2.0000i
A{4} =
    1
>> celldisp(B)
B{1,1} =
    1 1 1
B{2,1} =
    MATLAB
B{1,2} =
    0.8913 0.7621
B{2,2}{1} =
    1 2
    3 4
B{2,2}{2} =
    MATH2301
B{2,2}{3} =
    1.0000 + 2.0000i
B{2,2}{4} =
    1
```

To display the content of a component of a cell array:

```

>> A{1,2}
ans =
    MATH2301
>> B{2,2}
ans =
    [2x2 double] 'MATH2301' [1.0000 + 2.0000i] [1]
>> B{2,2}{2}
ans =
    MATH2301
>> B{2,2}{1}
ans =
     1 2
     3 4
>> B{2,2}{1}(2,2)
ans =
     4

```

A structure is a collection of objects labelled by field names. E.g.:

```

>> s = struct('name', 'harry', 'age', 45)
s =
    name: 'harry'
    age: 45
>> whos s
Name Size Bytes Class
s      1x1 266 struct array
Grand total is 8 elements using 266 bytes
>> s.name, s.age
ans =
    harry
ans =
    45
>> fieldnames(s)
ans =
    'name'
    'age'

```

We can also form array of structures:

```

>> t = struct('name', {'harry', 'sue', 'john'}, ...
    'age', {46, 37, 29})
t =
    1x3 struct array with fields:
    name

```

```
    age
>> t(2).name
ans =
    sue
>> t.age
ans =
    46
ans =
    37
ans =
    29
>> disp([t.age])
    46 37 29
```

1.4 Statement level control structures

1.4.1 Logical expressions

Logical expressions are statements that involve logical variables, relational operators and logical operators. The logical variables take value 1 (**true**) or 0 (**false**) in MATLAB. A logical variable can be created by using:

- The **true**, **false** functions.
- The **logical** function: converts numeric values to logical.
- Relational operators.
- Logical operators.
- All **is...** functions. E.g. **isinteger**, **isempty**, **isnan**, **isinf**, etc.

The use of **true/false** functions is listed below.

```
>> x = [true false]
x =
    1    0
>> A = true(2), B = false(2,3)
A =
     1     1
     1     1
B =
     0     0     0
     0     0     0
```

```
>> whos x A B
  Name Size Bytes Class
    A   2x2    4   logical array
    B   2x3    6   logical array
    x   1x2    2   logical array
Grand total is 12 elements using 12 bytes
```

The use of logical function is listed below.

```
>> a = 1;
>> b = logical(a); % convert a numeric value to logical
>> whos a b
  Name Size Bytes Class
    a   1x1    8   double array
    b   1x1    1   logical array
Grand total is 2 elements using 9 bytes
>> A = [1 2 0 ; -4 0 3 ];
>> B = logical(A)
Warning: Values other than 0 or 1 converted to logical 1.
B =
     1     1     0
     1     0     1
>> whos A B
  Name Size Bytes Class
    A   2x3   48   double array
    B   2x3    6   logical array
Grand total is 12 elements using 54 bytes
```

The following table list the relational operators supported by MATLAB.

Operator	Description
==	equal
~=	not equal
<	less than
<=	less than or equal
>	greater than
>=	greater than or equal

```
>> 3 > 4
ans =
     0 % 0 when relation is false
>> s = 2 < 3 % assign the answer to variable s.
s =
```

```

    1 % 1 when relation is true
>> u = [1 3 9 0]; v = [2 1 9 3]; % u and v same size
>> s1 = u ~= v, s2 = u >= v
s1 =
    1 1 0 1
s2 =
    0 1 1 0
>> A = [2 3 ; 4 5]; B = [2 4 ; 5 5]; % A and B same size
>> C = A ~= B
C =
    0 1
    1 0
>> whos s1 s2 C
Name Size Bytes Class
    C    2x2     4 logical array
    s1    1x4     4 logical array
    s2    1x4     4 logical array
Grand total is 12 elements using 12 bytes

```

The following table list all logical operators. Note: A, B, C are arrays of same size.

Operator	Description
$\&$	$C = A \& B, C(i, j) = \begin{cases} 1, & A(i, j) \neq 0 \text{ and } B(i, j) \neq 0 \\ 0, & \text{otherwise} \end{cases}$
$ $	$C = A B, C(i, j) = \begin{cases} 1, & A(i, j) \neq 0 \text{ or } B(i, j) \neq 0 \\ 0, & \text{otherwise} \end{cases}$
$\sim$	$C = \sim A, C(i, j) = \begin{cases} 1, & A(i, j) = 0 \\ 0, & \text{otherwise} \end{cases}$
any	$v = any(u), v = \begin{cases} 1, & \text{at least one } u(i) \neq 0 \\ 0, & \text{otherwise} \end{cases}$
all	$v = all(u), v = \begin{cases} 1, & \text{all } u(i) \neq 0 \\ 0, & \text{otherwise.} \end{cases}$

```

>> A = [2 0 3 ; 5 0 1]; B = [0 0 -3 ; 1 3 0];
>> C = A & B, D = A | B, E = ~A,
C =
    0 0 1
    1 0 0
D =

```

```

    1 0 1
    1 1 1
E =
    0 1 0
    0 1 0
>> u = [2 0 1];
>> v1 = any(u)
v1 =
    1
>> v2 = all(B) % v2 = [all(B(:,1)) all(B(:,2)) all(B(:,3))]
v2 =
    0 0 0

```

The `is...` functions: `isequal`, `isempty`, `isfinite`, `isinf`, `isinteger`, `isprime`, `isnan`
 ...

```

>> x1 = []; x2 = [1 2];
>> v1 = isempty(x1), v2 = isempty(x2)
v1 =
    1
v2 =
    0
>> A = [2 0 3 ; 5 0 1]; B = [0 -3 ; 1 3]; C = [2 0 3; 5 1 1];
>> v1 = isequal(A,A), v2 = isequal(A,B), v3 = isequal(A,C)
v1 =
    1
v2 =
    0
v3 =
    0

```

Logical arrays are used in array indexing and in conditional statements `if`, `elseif`.

Example 1.4.1 Find all elements of $\mathbf{x}$ which are < 5 and ≥ 0 . Set new value 5 to those elements.

```

>> x = [5 2 8 4 9 0 -1];
>> v = x < 5 & x >= 0
v =
    0 1 0 1 0 1 0
>> x(v)
ans =
    2 4 0
>> x(v) = 5 % same as x(x<5&x>=0) = 5

```



```
x =
    5  5  8  5  9  5 -1
```

The zero values cannot be used to index a vector or a matrix.

```
>> v = [0 1 0 1 0 1 0];
>> x(v)
??? Subscript indices must either be real positive
integers or logicals.
>> A = [2 0 3 ; 5 0 1]; B = [0 0 -3 ; 1 3 0];
>> A(A&B) = inf
A =
     2     0   Inf
   Inf     0     1
>> A = magic(3), A(~isprime(A)) = 0
A =
     8     1     6
     3     5     7
     4     9     2
A =
     0     0     0
     3     5     7
     0     0     2
```

1.4.2 Conditional statements

Conditional statements or `if` statements provide a mechanism for branching in a computer program, i.e. they make it possible to execute selectively on the basis of information generated while the program is running. In MATLAB, an `if` statement takes the following form:

```
if <logical expression 1>
    <statement 1>
elseif <logical expression 2>
    <statement 2>
else
    <statement 3>
end
```

Zero or more than one `elseif` branch is permitted. The `else` branch can also be omitted.

Example 1.4.2 Let $\mathbf{x}$ be given. For each element of $\mathbf{x}$, if it is nonpositive then negate it, otherwise just square it.

```
>> x = [-1 2 5 -3];
>> if x <= 0
    y = -x;
else
    y = x.^2;
>> end
>> disp([x;y])
    -1     2     5    -3
     1     4    25     9
```

1.4.3 Loop statements

A loop is a programming construct in which a group of statements is executed repeatedly. In MATLAB, loops are of two kinds: **for** loops and **while** loops. A **for** loop has the form

```
for n = 1:N % a value for N has to be pre-defined
    <statement>
end
```

Example 1.4.3 The following commands

```
>> for n = 1:5
    disp([num2str(n) '^2 = ' num2str(n^2)])
end
```

will produce

```
1^2 = 1
2^2 = 4
3^2 = 9
4^2 = 16
5^2 = 25
```

Note: the command `num2str` converts numbers to a string.

Example 1.4.4 To find the unique whole number K such that $(K - 1)^2 \leq 1073 < K^2$, we could use the following script:

```
for n = 1:40
    if n^2 > 1073
        break
    end
end
disp(['K = ' num2str(n)])
disp([num2str(n-1) '^2 = ' num2str((n-1)^2)])
disp([num2str(n) '^2 = ' num2str(n^2)])
```

Example 1.4.5 What is the output of the following commands?

```
>> A = [8 1 6 ; 3 5 7; 4 9 2];
>> for J = 1:3
    for I = 1:3
        if (I == J)
            A(I,J) = 2;
        elseif abs(I-J) == 1
            A(I,J) = -1;
        else
            A(I,J) = 0;
        end
    end
end
>> A
```

A while loop has the form

```
while <logical expression>
    <statement>
end
```

The for loop in the last example could be replaced with the following while loop:

```
n = 1;
while n^2 < 1073
    n = n + 1;
end
disp(['K = ' num2str(n)])
disp([num2str(n-1) '^2 = ' num2str((n-1)^2)])
disp([num2str(n) '^2 = ' num2str(n^2)])
```

Obviously, the statements in a while loop must modify the value of the logical expression, so that the latter eventually becomes false and the loop terminates. There is always the danger of an infinite loop, in which the logical expression never becomes false, but is always true, perhaps due to programmer not considering all possibilities. The for ... break is safer.

1.5 Functions

1.5.1 Built-in functions

MATLAB has numerous built-in functions. Use the command `help elfun` to see available elementary mathematical functions. Scalar functions naturally take scalar arguments; but will also act on matrices elementwise.

```
>> x = 1.2; y = [1:0.2:2]; X = [1 2 ; pi 5];
>> sin(x), sin(y), sin(X)
ans =
    0.9320
ans =
    0.8415  0.9320  0.9854  0.9996  0.9738  0.9093
ans =
    0.8415  0.9093
    0.0000 -0.9589
```

Vector functions act on vectors and matrices.

Example 1.5.1 If v is a row vector or column vector, `sum(v)` gives the sum of the elements of v . The command `sum(v,1)` treats v as a matrix and adds its components columnwise. The command `sum(v,2)` finds the sum of elements in each row.

```
>> v = [1:5];
>> s1 = sum(v), s2 = sum(v,1), s3 = sum(v,2)
s1 =
    15
s2 =
    1  2  3  4  5
s3 =
    15
```

The commands `prod(v)`, `prod(v,1)`, and `prod(v,2)` have the same meaning for products of elements of v .

Matrix functions take matrix arguments. For example, given a square matrix A , the command `det(A)` computes the determinant of A , `inv(A)` computes the inverse of A , `eig(A)` computes the eigenvalues of A . Let B be a 4×3 matrix, the command `reshape(B,2,6)` will take elements of B columnwise and form a new 2×6 matrix.

```
>> A = [1 3 5 ; 2 4 6 ; 9 2 1];
>> det(A)
ans =
   -12
>> inv(A)
ans =
    0.6667   -0.5833    0.1667
   -4.3333    3.6667   -0.3333
    2.6667   -2.0833    0.1667
>> eig(A)
ans =
   10.8674
```

```

-5.0846
0.2172
>> B = [1 3 5 ; 2 4 6 ; 9 2 1 ; 4 8 2];
>> reshape(B,2,6)
ans =
    1    9    3    2    5    1
    2    4    4    8    6    2

```

1.5.2 Function files

Function files (which are M-files) allow one to create one's own MATLAB commands. A function file usually has these lines:

```

function [yout1, yout2, ...] = funcname(xin1, xin2, ...)
% comment line 1
% comment line 2
% ...
yout1 = ...;
yout2 = ...;
...

```

Notes:

- The function M-file name and the function name that appears in the first line of the file should be identical. In reality, MATLAB ignores the function name in the first line and executes functions based on the file name.
- All comment lines immediately following the function definition line are displayed by help funcname.
- A single output does not require []. E.g. `function y = funcname(xin1,xin2).`
- Input variable names given in the function definition line are local to the function. An input variable name can be the name of another function. On execution, this name must be passed on as a character string, i.e., enclosed within two single right quotes.
- All variables inside a function are local and are erased after execution of the function, whereas those in a script file are left in the MATLAB workspace after execution of the script.
- Functions can have arguments, script files do not.
- Inside the body of a user-defined function, `nargin` and `nargout` return the numbers of declared input and output arguments. `nargin('funcname')` and `nargout('funcname')` return those values for the M-file function `funcname.m`.

Example 1.5.2 Write a MATLAB function defining the following mathematical function:

$$f(x) = \begin{cases} 0 & \text{for } x < -2 \\ (x+2)^2/4 & \text{for } -2 \leq x < 0 \\ (3-x)/3 & \text{for } 0 \leq x < 3 \\ 0 & \text{for } x \geq 3 \end{cases}$$

```
function y = piecefunc(x)
% EXAMPLE of a piecewise-defined function
%
%      (x+2)^2/4   for -2 <= x < 0,
% y = (3-x)/3     for 0 <= x < 3,
%      0          otherwise
y= zeros(size(x)); % preallocate y
i1 = find(-2 <= x & x < 0)
i2 = find( 0 <= x & x < 3)
y(i1) = (x(i1)+2).^2/4;
y(i2) = (3-x(i2))/3;
```

Example 1.5.3 Write a function file to define the probability density function

$$f(x) = \frac{1}{\sqrt{2\pi}\sigma} \exp\left(\frac{-(x-\mu)^2}{2\sigma^2}\right).$$

```
function density = normal_dist(x,mu,sigma)
% function density = normal_dist(x,mu,sigma)
% computes the probability density function for a normally
% distributed random variable with mean mu and standard
% deviation sigma. If the argument mu and sigma are omitted,
% then a standard normal distribution is assumed
% (mu = 0 and sigma = 1).
%
if nargin == 1
    mu = 0;
    sigma = 1;
end
density = exp(-0.5* ((x-mu)/sigma).^2)/(sqrt(2*pi)*sigma);
```

The following script file will illustrate the use of `normal_dist`.

```
% Script file exec_normal_dist.m
% using the function normal_dist, with 1 and 3 input
% arguments
```

```

x = linspace(-4,4);
y1 = normal_dist(x);
y2 = normal_dist(x,1,0.5);
plot(x,y1,x,y2)
legend('standard normal','mu=1, sigma=0.5')
trapz(x,y1) % trapezoidal rule to evaluate integral
trapz(x,y2) % type "help trapz" to learn more

```

1.5.3 Anonymous functions

We can also define an anonymous function either at the MATLAB command line:

```

>> fun1 = @(x) x.^2;
>> fun2 = @(x,y) x+y;

```

or in an M-file (script file or function M-file):

```

sqr = @(x) x.^2;
x = linspace(0,2);
plot(x,sqr(x));

```

1.5.4 Passing functions as input

A function handle is a MATLAB value that provides a mean of referring to a function indirectly. For example, if

```
sqr = @(x) x.^2
```

then `sqr` is the handle of the anonymous function x^2 . If a function is defined in a function M-file, then a handle to this function can be defined by

```
funhandle = @filename.
```

Some MATLAB built-in functions (e.g. `fzero` and `feval`) or user-defined functions have as input another function. The function handle is used to pass on this input.

With the function `sqr` defined above we have

```

>> feval(sqr,2) % This is the same as sqr(2)
ans =
     4

```

For functions defined in function M-files like `piecefunc` in Example 1.5.2, there are several ways to pass it on as an input of another function.

```
>> piecefunc_handle = @piecefunc;
>> feval(piecefunc_handle,2)
ans =
    0.3333
>> feval(@piecefunc,2)
ans =
    0.3333
>> feval('piecefunc',2)
ans =
    0.3333
```

1.6 Graphics

1.6.1 2-D graphics

Example 1.6.1 Plot $y = \exp(x)$ for $0 \leq x \leq 3$ using the `plot` command.

```
% create some points on the domain [0,3]
x = 0:0.3:3;
% compute function values at x
y = exp(x);
plot(x,y)
% We may need more points to get a smooth plot
% create 100 equally spaced points on [0,3]
x = linspace(0,3);
y = x.^2 .* exp(x);
plot(x,y)
```

Different linestyles (+ * -), and colours (y m c r g b w k) can be chosen in the options of the `plot` command. Type `help plot` for more information.

```
>> x = linspace(0,1); y = log(x); plot(x,y,'g+')
```

The `fplot` command selects points on the x-axis automatically.

```
fplot('sin(10*x)', [0 3])
fplot('x^2 * exp(x)', [-1,1])
% or
fplot('x.^2 .* exp(x)', [-1,1])
% Change axes. Type help axis for more info
axis([0 0.5 0 1])
```

The following commands add labels to the axes, title and legend to the plot: `xlabel`, `ylabel`, `title`, `text`, `legend`. Use `help` for more information.

Example 1.6.2 Plot the Bessel functions of order zero, one, and two on $[0, 10]$. (Bessel functions of order μ are solutions of the ODE $x^2 y'' + xy' + (x^2 - \mu^2)y = 0$. Type `help besselj` to get more info.)

```
% This script plots 3 Bessel functions of order 0,1,2 on
% [0,10]. Type "help besselj" for more info.
close all
figure(2)
x = [0:0.2:10];
y0 = besselj(0,x); y1 = besselj(1,x); y2 = besselj(2,x);
plot(x,y0,x,y1,x,y2)
legend('\mu=0', '\mu=1', '\mu=2') % \mu to produce Greek letter mu
xlabel('x')
ylabel('B_\mu(x)') % underscore _ to produce subscript
title('Bessel functions B_\mu')
```

Another way to plot multiple graphs in one figure window is to use `hold on`:

```
>> x = linspace(0,2); y1 = x.^2; y2 = sin(x);
>> plot(x,y1)
>> hold on
>> plot(x,y2)
>> hold off
```

To save a figure: use `print`

```
>> print -dps filename % save as .ps file
>> print -deps filename % save as .eps file
>> print -depssc filename % save as .eps file (colour)
>> print -djpeg filename % save as .jpeg file
```

Note: MATLAB 2014b seems to have some problems with exporting Greek symbols to `eps` files. The quick fix is to use the following commands (taking the previous plot as an example)

```
>> title('Bessel function $B_\mu$', 'interpreter', 'latex');
>> print -depssc2 filename
```

Example 1.6.3 Plot $\sin(x)$ and $\cos(x)$ in one figure for $a \leq x \leq b$, where a and b are input parameters.

```
%See file plot_sincos.m
a = input('a = '); % prompt for user input
b = input('b = '); % prompt for user input
if a >= b
```

```

    error('a must be strictly less than b')
end
x = [a:0.1:b];
y1 = sin(x);
y2 = cos(x);
plot(x,y1,x,y2)
legend('sin x', 'cos x')
print -deps sincos

```

The `error` built-in function displays the message, exits the M-file, and returns control to the keyboard.

Example 1.6.4 Plot the Riemann zeta function $\zeta = \sum_{n=1}^N n^{-s}$ for $s \in [a, b]$ and $a > 1$.

```

disp('Plot zeta(s) for a < s < b where a > 1')
a = input ('a = ');
b = input ('b = ');
if a <= 1
    error('a must be strictly greater than 1')
elseif b <= a
    error('b must be strictly greater than a')
end
s = linspace(a,b);
N = input('Number of terms = ');
n = [1:N];
mat1 = repmat(n',1,length(s));
mat2 = -repmat(s,length(n),1);
y = sum(mat1 .^ mat2);
plot(s,y)
xlabel('s'), ylabel('zeta(s)');

```

In the previous example, we have used the `repmat` command. The actions of the command can be seen in the following code

```

>> a=2; b=4; s=linspace(2,4,3);
>> N=5; n = [1:N];
>> mat1 = repmat(n',1,length(s))
mat1 =
    1 1 1
    2 2 2
    3 3 3
    4 4 4

```

```

5 5 5
>> mat2 = -repmat(s,length(n),1)
mat2 =
    -2    -3    -4
    -2    -3    -4
    -2    -3    -4
    -2    -3    -4
    -2    -3    -4

```

Parametric curves can be drawn as follows:

```

>> t = linspace(-pi,pi);
>> plot(cos(t),sin(3*t))
>> xlabel('x(t)') % x-axis label
>> ylabel('y(t)') % y-axis label
>> title('Parametric curve (cos(t),sin(3t))')

```

Curves can be drawn using polar coordinates:

```

>> theta = linspace(-pi,pi,300);
>> r = 1 ./ (1+0.6*cos(10*theta));
>> polar(theta,r)

```

Sometimes, we need to put many plots in the same figure. The command **subplot** will help us to achieve this. The figure window can be divided into rectangular panes that are numbered rowwise. Plots are then output to each pane.

```

x = linspace(0,pi);
y1 = sin(x);
y2 = x.^2 ./ (1+x.^3); % note the dots
subplot(1,2,1) % divide the graphic window into 2 panes,
                % 1 row and 2 columns, and get the figure
                % in the first pane
plot(x,y1)     % plot in the first pane
subplot(1,2,2) % get the figure in the 2nd pane
plot(x,y2)     % plot in the second pane

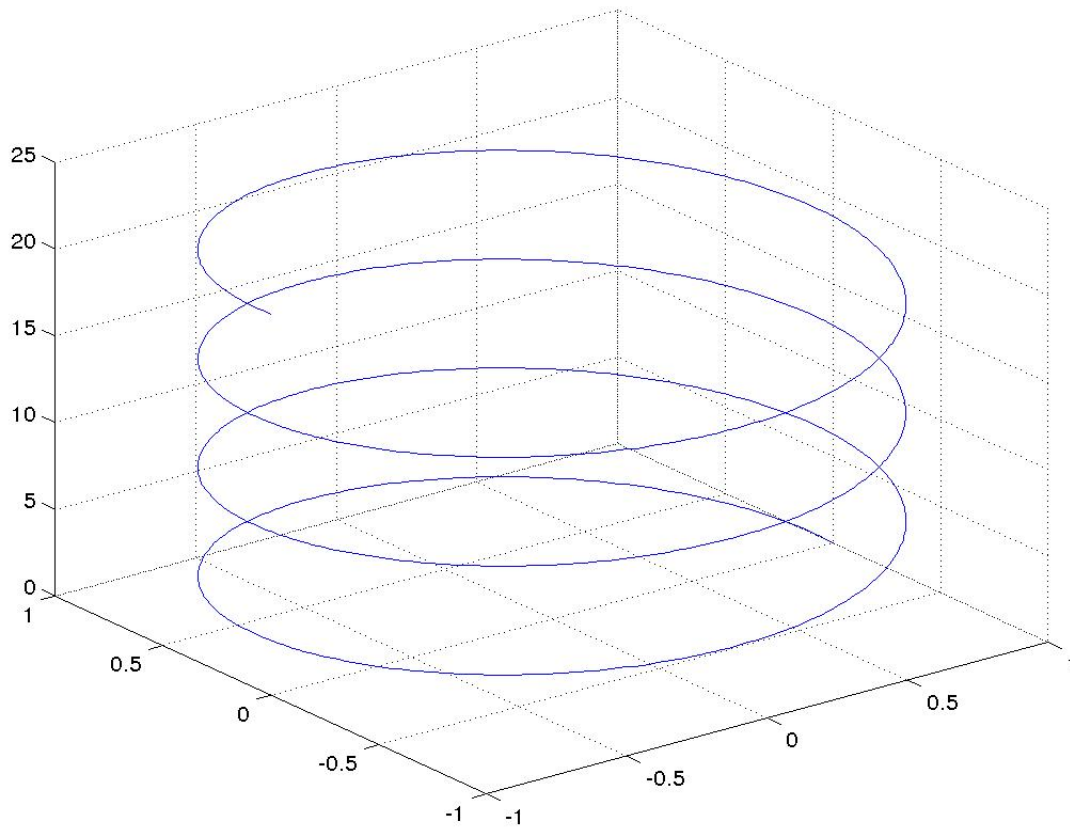
```

1.6.2 3-D graphics

Type **help graph3d** to obtain a list of functions available for 3D graphics. Colors can be used to represent the 4th dimension or values of functions defined on surfaces. The most commonly used commands for 3D graphics are: **plot3**, **mesh**, **surf**. Graphics can be rotated by using the **Rotate 3D** button on the figure menu, or using the **view** command. Type **help view** for more details.

Example 1.6.5 Plot the circular helix $x = \cos t, y = \sin t, z = t$ for $t \in [0, 7\pi]$.

```
>> t=linspace(0,7*pi,1000);
>> plot3(cos(t),sin(t),t);
>> axis on
>> grid on
```



To plot a surface in R^3 , firstly we need to create a grid via the `meshgrid` command.

```
>> x = -2:2; % create the vector x = [-2 -1 0 1 2]
>> y = -1:1; % create the vector y = [-1 0 1]
>> [X,Y] = meshgrid(x,y) % create a grid of 15 points
% their x-coordinates are stored in X
% their y-coordinates are stored in Y
X =
    -2    -1     0     1     2
    -2    -1     0     1     2
    -2    -1     0     1     2
```

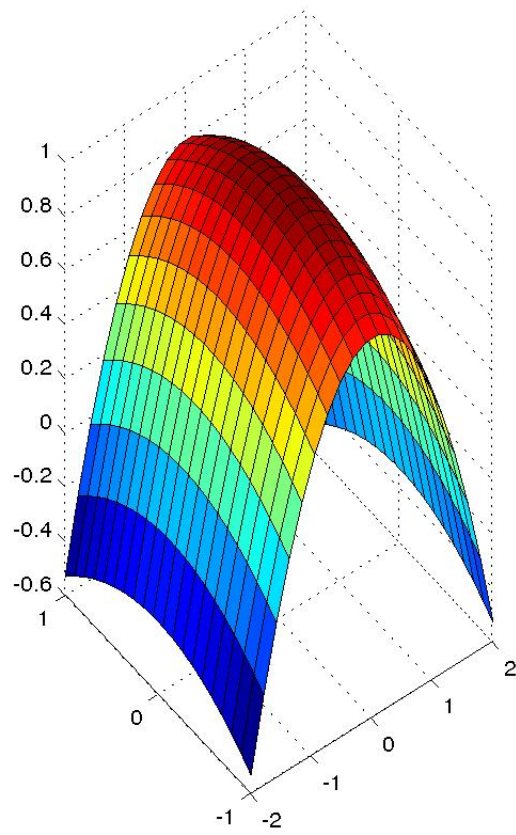
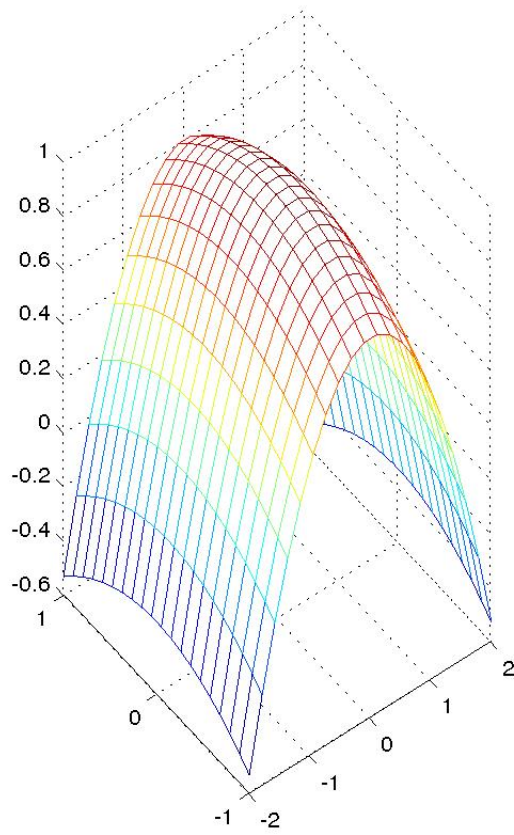
```
Y =
    -1    -1    -1    -1    -1
     0     0     0     0     0
     1     1     1     1     1
```

In general X, Y are matrices of size $n \times m$ where m and n are the lengths of x and y , resp, and $X(i, j) = x(j)$, $Y(i, j) = y(i)$.

In the following example, we will plot surfaces in 3D using `mesh` and `surf` commands.

Example 1.6.6 Plot the surface $z = 1 - x^2/3 - y^2/5$ for $-2 \leq x \leq 2$ and $-1 \leq y \leq 1$. The surface is determined by points $(X(i, j), Y(i, j), Z(i, j)) = (x(j), y(i), Z(i, j))$.

```
>> % see plotS.m
>> x = linspace(-2,2,20);
>> y = linspace(-1,1,20);
>> [X,Y] = meshgrid(x,y);
>> Z = 1 - X.^2/3 - Y.^2/5;
>> subplot(1,2,1)
>> mesh(X,Y,Z) % or mesh(x,y,Z)
>> subplot(1,2,2)
>> surf(X,Y,Z) % or surf(x,y,Z)
```



1.7 Efficiency

Performance of your MATLAB code can be improved by vectorising loops and preallocating arrays.

1.7.1 Vectorising loops

MATLAB is a matrix language designed for vector and matrix operations. To speed up your M-file code you should use vectorising algorithms that take advantage of this design. Whenever possible, convert for and while loops to vector or matrix operations.

Test the following code on your system. In the code, `cputime` is used to compute the difference in time. `cputime` is a built-in function that returns the CPU time (in seconds) that has been used by the MATLAB process since MATLAB started.

Here is one way to compute the sine of 100001 values ranging from 0 to 10:

```
t0 = cputime;
i = 0;
for t = 0:.01:1000
    i = i + 1;
    y(i) = sin(t);
end
t1 = cputime;
loop_time = t1-t0
```

The vectorised version of the same code is

```
t0 = cputime;
t = 0:.01:1000;
y = sin(t);
t1 = cputime;
vec_time = t1-t0
```

1.7.2 Preallocating arrays

A loop that incrementally increases the size of a data structure each time through the loop can adversely affect performance and memory use. Repeatedly resizing arrays often requires that MATLAB spend extra time looking for larger contiguous blocks of memory and then moving the array into those blocks. If you cannot vectorise a piece of code, you can make your loops go faster by preallocating any vector in which output results are stored.

Compare the following two pieces of code, assuming that a value for n is given in advance.

```
% Without preallocation
clear x
for j=1:n
    for i=1:n
        x(i,j)=(i+j)/n;
    end
end
% With preallocation
clear x
x=zeros(n,n); % preallocate x
for j=1:n
    for i=1:n
        x(i,j)=(i+j)/n;
    end
end
```